Documentation

Cette documentation est à l'usage des utilisateurs susceptibles de réaliser un exercice.

Généralités

3 zones de codes

- **options :** on y indique des variables définissant le mode d'exécution de l'exercice.
- **initialisation :** on y initisalise les variables aléatoires qui rendent l'exercice unique.
- **code :** on y place le code servant à l'exécution de l'exercice.

Commentaires

Une ligne vide est ignorée. Tout ce qui suit `#` est considéré comme commentaire

Code à balises

On utilise une langage fait de balises, comme en html.

- Certains balises fonctionnent par paire : `<bloc>...</bloc>`
- D'autres balises sont auto-fermantes : `<bloc/>`
- En général, il est possible d'indiquer une option avec l'ouverture du bloc : `<bloc:info/>` ou `<bloc:info>...</bloc/>`

Les options

Dans le cadre des options, on ne déclarera que des blocs `<option>`

```
<option:nom>
key1 => value1
key2 => value2
key3 => value3
</option>
```

- `nom` est le nom de l'option et sera accessible ensuite avec `@nom`
- `key1` est une clé et doit être numérique. C'est elle qui sera stockée en BDD quand l'utilisateur voudra indiquer l'option choisie.
- `value1` est la valeur correspondante. Elle sera affichée dans le menu de chois des options.

Remarque : On refuse les noms d'options commençant par `_`.

Important : La première valeur de l'option est toujours la valeur par défaut.

Initialisation et code

Initialisation

L'exécution d'un exercice peut se faire dans deux contextes :

- il s'agit d'une nouvelle exécution, il faut donc initialiser les paramètres utiles.
- Il s'agit de l'exécution d'un exercice préalablement initialisé et dont les paramètres ont été stocké en BDD.

L'initialisation ne s'exécute que dans le premier cas. Ainsi, réexécuter un exercice précédemment sauvegardé en BDD devrait toujours donner le même résultat.

Affectation

C'est l'essentiel du travail d'initialisation : calculer une valeur pour les paramètres constitutifs de l'exercice. Pour l'initialisation, ces calculs utilisent souvent de l'aléatoire.

Pendant l'exécution de l'exercice, on aura également besoin de faire des calculs par exemple pour calculer les réponses attendues.

Nommage des variables

- On annonce les variables en les faisant précéder par `@`.
- L'affectation d'une variable se fait avec un signe `=`
- Les variables dont le nom commence par `_`, par exemple `@_x`, définies dans le bloc d'initialisation ne seront pas sauvegardées et seront donc perdues au moment de l'exécution de l'exercice. Elles peuvent servir de variable intermédiaire pour faciliter la rédaction de l'init.

Important : *On interdit les noms de variables commençant par* `__`.

Formule ordinaire

On peut affecter une variable avec une formule de calcul ordinaire qu'il suffit d'écrire.

Exemple :

```
@_x = 3
@y = 3+2*@_x^2
```

On pourra ainsi utiliser

- les symboles habituels `+`, `-`, `*`, `/`,
- puissance avec `^` ou `**`, le symbol `²` est reconnu
- `cos`, `sin`, `tan`, `atan`, `exp`, `log`, `ln`, `tan`, `sqrt`, `abs`
- `mod(x;y)` pour modulo et `div(x;y)` pour la division entière *Attention à bien uttiliser ;*
- `diff(expression;x)` permet de calculer une dérivée *; aussi !*
- `pgcd(x;y)` et `ppcm(x;y)` ou même `pgcd(x;y;z)`
- `min(x;y;z...)`, `max(x;y;z...)`

Il est également possible de forcer une représentation de variable en demandant un formatage. Ainsi dans le code ci-dessous, on substitue `pi` par son écriture décimale à 3 chiffres après la virgule.

```
@x = 5 + {pi:3f}
```

Autre exemple, supposons que l'on veuille $x = \frac{a \times d}{b \times d}$ dane le cadre d'un exercice de simplification de fraction. On voudrait alors que les calculs $a \times b$ et $b \times d$ soient effectués mais pas la simplification de la fraction. La formule ci-dessous permet de forcer cette écriture.

```
@x = {@a*@d:s}/{@b*@d:s}
```

Évaluation d'une pile

Si l'expression à évaluer est entourée de `<P: >`, alors elle est comprise comme une pile. L'expression est alors découpées selon les espaces.

Les éléments de la pile subissent un prétraitement :

- Si un bloc est reconnu de la forme `@name` ou `@name.sub` ou `@name[index]` alors il est remplacé par la valeur correspondante. Cela pourrait donc être un objet.
- Si une chaîne contient des éléments de la forme `@name` ou `@name.sub` ou `@name[index]` alors ils sont substitués mais alors sous forme textuelle et avec des parenthèses !

  Par exemple si on a `3*@b+2` et que `@b` contient `x+4` alors on obtiendra `3*(x+4)`

On suite on passe à l'exécution de la pile :

- Tout bloc non chaîne de caractère est considéré comme opérande est placé dans la pile des opérandes.
- Tout bloc de la forme `"module.fonction"` est considéré comme une fonction.
  - Cette fonction sera cherchée parmi les modules disponibles. Il faudra donc qu'elle existe bien.
  - Cette fonction a une certaine **arité**. Elle prendra donc les opérandes dans la pile des opérandes. Il faut qu'il y en ait assez.
  - Attention à l'ordre : l'opérande le plus haut sur la pile sera celui donné en dernier à la fonction.
  - Le résultat de l'exécution est placé sur la pile des opérandes.

Affectation d'un tableau

On peut affecter un tableau simple (pas un tableau de tableau) avec la syntaxe habituelle `[...]`.

```
@t = [1, 2, 3]
@v = [@a, @c-2, 17]
```

Commandes plus avancées pour des tableaux

Affectation `<:n>`

- `@a <:3>= 5` va ainsi créer un tableau `[5, 5, 5]`.
- On dispose de la variable `@__i` qui représente l'index. Ainsi :

  `@a <:3>= 5 + @__i` va affecter `[5, 6, 7]`

- On peut également faire des calculs sur un tableau :

  `@b <:3>= @a[]^2` va affecter `[25, 36, 49]`.

Attention, l'utilisation dans l'expression de `@a[]` lèvera une erreur...

- si on a pas précisé `<:3>` ou une autre taille avant le `=` pour indiquer que l'on fait un calcul sur tableau,
- si la taille indiquée excède la taille de `@a`.

Avec une pile

On peut aussi travailler avec une taille non connue à l'exécution. Ceci par exemple est autorisé :

```
@n = <P:3 10 Alea.entier>
@a <:@n>= @__i^2
```

Ce qui créera une tableau de taille aléatoire.

On peut naturellement utiliser ces affectations dans le cas d'une pile.

```
@n = <P:3 10 Alea.entier>
@a <:@n>= <P:0 100 Alea.entier>
```

Le répéteur est un tableau `<:@a>`

Ce qui crée un tableau d'entiers aléatoires de taille aléatoire.

Le répéteur peut être un tableau. Par exemple :

```
@a <:5>= @__i + 10
@b <:@a>= @__v^2 + @__i + 1
```

Ainsi le tableau `@b` est obtenu en parcourant les items de `@a`.

On dispose du raccourci `@__v` désignant l'item de `@a` en cours de lecture et `@__i` désignant l'indice.

Ici, `@a = [10, 11, 12, 13, 14]` et donc on prend chaque item, on l'élève au carré, on lui ajoute son indice et on ajoute `1` ce qui donne `@b = [101, 123, 147, 173, 201]`.

Affectation push

On peut également définir un tableau par ajouts successifs :

```
@a[] = 3
@a[] = 12
```

Crée un tableau contenant les valeurs `3` et `12`.

Arrondi

Si l'on fait suivre le `=` d'un nombre `~n` on force une évaluaiton arrondie à n décimales.

```
@x =~2 8/3
```

La valeur affectée à `@x` sera `2.67`

Alea entier

On dispose de la fonction `Alea.entier` mais cela nécessite de passer par la pile. Ainsi, quand on veut créer un nombre entier aléatoire entre `0` et `5` compris par exemple, il faut écrire :

```
@x = <P: 0 5 Alea.entier >
```

Cela peut sembler un peu lourd. Alors on a la possibilité d'utiliser une variable réservée : `@__a._6` qui crée un nombre entier aléatoire de `0` à `6` **exclus**.

```
@x = @__a._6
```

Naturellement on peut choisir n'importe quel nombre entier strictement positif.

Attention : si un alea est plusieurs fois présent dans le même calcul, ce ne sera pas le même !

```
@x = @__a._6 + @__a._10 + @__a._6
```

Le premier et le dernier nombre aléatoire, choisi dans $[0;6[$, ne sont pas les mêmes !

Comme les mêmes besoins reviennent constamment, il existe divers formats de nombres aléatoires :

- `@__a._10` ou `@__a.i10`, alea entier de 0 à 10 exclu,
- `@__a.I10`, alea entier de 1 à 10 inclu,
- `@__a.f10`, alea flottant de 0 à 10 exclu
- `@__a.s10`, alea entier de -10 à 10 exclus (de -9 à 9)
- `@__a.S10`, alea entier de -10 à 10, inclus et sans le 0.
- `@__a.P10`, renvoie un premier aléatoire jusqu'à 10 compris. La borne peut aller de 2 à 1008 compris (car 1009 le plus grand premier de la liste prédéfinie est 997 et que le suivant est 1009)
- `@__a.vxyz`, le `v` signale qu'une lettre sera choisi aléatoirement parmi `xyz`. Liste des variable au choix, par ex `@__a.vtAMnK`... Permet de varier les énoncés et de ne pas toujours utiliser `x` comme variable.

Infini

On peut gérer une quantité infini avec `infinity` ou `inf` ou `infini` ou `∞`.

Chaîne de caractères

Il peut être utile de stocker une chaîne pour moduler des textes en fonction des paramètre d'un exercice. Par exemple, supposons que j'ai un taux `t` en % et que je veuille adapter le texte en disant **augmente** ou **dimminue** selon le signe de `t`.

```
<if @t > 0>
  @taff = @t # taux affiché
  @texte = "augmente"
<else>
  @taff = -@t
  @texte = "diminue"
<endif>
<texte>
On considère une quantité qui {@texte:} de ${@taff:f}\,\%$ par mois.
</texte>
```

Ici, `{@texte:}` se contente de remplacer par le contenu entre les `" "`. On pourrait donc très bien faire ceci :

```
<if @__a.I2 == 1>
@var = "x"
<else>
@var = "t"
<endif>
@formule = 3*@var^2 - 5@var + 1
```

Voyons différents cas :

```
@a = "x"      # est une chaîne
@b = @a       # @b est une copie de @a, donc est une chaîne
@c = 3 + @a   # @a remplacé par sa valeur texte et @c est expression 3+x
@d = "3 + @a" # @d est la chaîne de caractère "3 + x"
```

Ensemble

On peut définir des ensembles sous forme d'intervalles.

```
@a = __inter__ ]15;inf[
```

Le préfixe `__inter__` annonce qu'il faudra analyser la quantité en tant qu'intervalle.

- On n'autorisera pas de variables dans les bornes, mais on a le droit à des expression comme $\frac{\pi+\sqrt{2}}{4}$
- L'intervalle doit être dans le bon sens
- Les bornes sur infini doivent être ouvertes

On pourra ensuite faire des calculs. On dispose pour cela des opérateurs :

- Union : `∪` ou encore | (qui est sur le clavier) ou `union`
- Intersection : `∩` ou encore `&` (qui est sur le clavier) ou `inter` ou `intersection`
- il faut annoncer par `__inter__` le calcul d'intervalles.

```
@A = __inter__ ]-10;inf[
@B = __inter__ [0;35[
@C = __inter__ @A & @B
@D = __inter__ @A | @B
```

Le résultat est immédiatement calculé.

On dispose également de `∅` pour ensemble vide. On pourra écrire `0` ou `vide` ou `empty`

Contrôle de flux

Dans le bloc d'**initialisation** ou le bloc de **code** de l'exercice, le concepteur a le droit à quelques structures de contrôle de flux, c'est à dire des `if`, `else`... Ces structures supposent la possibilité d'énoncer des **conditions logiques**.

Conditions

Comparaison

Il est possible de comparer des quantités en utilisant `==`, `!=`, `<`, `<=`, `>`, `>=`.

Par exemple : `@a <= 3`

exists

Quand on conçoit un exercice avec options, selon les options, on peut définir divers paramètres. Il est donc utile de pouvoir utiliser quels paramètres sont définis ou pas.

On peut donc utiliser un opérateur `exists`.

Ainsi ce test est valide : `@a exists`. Ce test renvoie vraie si `@a` a bien été défini et faux sinon. On peut utliser le opérateurs logiques (voir ci-dessous), par exemple `@a exists and @b exists`.

Opérateurs logiques

On peut structurer la condition logique avec les opérateurs `and` et `or`.

Le `and` est prioritaire.

En cas d'opération complexe, il conviendra de faire usage de `{...}` en guise de parenthèses.

Par exemple, cette condition est autorisée :

```
{@a == 0 or @a == 2} and @b = 10
```

some et all

Dans le cas d'un tableau, il est possible de chercher si la totalité des éléments vérifient une condition ou si certains la vérifient. On utilisera alors les opérateurs `all` et `some`.

Par exemple, si `@a = [1, 4, 5, 10]` on pourra faire une comparaison pour savoir si un des éléments est égal à 5 ou encore si tous les éléments sont égaux à 5.

- Au moins un élément égal à 5

```
some {@a==5}
```

- Tous les éléments égaux à 5

```
all {@a==5}
```

- Au moins un élément égal à 5 et tous inférieurs à 10

```
some {@a == 5} and all {@a < 10}
```

- Au moins un entre 5 et 10

```
some {@a > 5 and @a < 10}
```

if

structure de `if` classique.

```
<if @a == @b>
...
<elif @a != 12>
...
<else>
...
<endif>
```

Les `if` peuvent être imbriqués.

On peut utiliser `</if>` pour fermer le bloc.

needed

Indique une contrainte.

Si elle n'est pas respectée pendant l'initialisation, celle-ci redémarre automatiquement pour un nouvel essai.

L'idée est d'initialiser aléatoirement et de « relancer les dés » jusqu'à obtenir des valeurs convenables.

```
<needed @xA != @xB or @yA != @yB >
```

Donc, un échec de `needed` interrompt la tentative d'initialisation et en relance une autre à partir de zéro. Pour éviter une situation bloquante, au bout d'une centaine d'essai, la tentative d'initialisation est abandonnée et un message d'erreur est envoyé.

Noter que, bien que `needed` soit un bloc singleton, il ne faut pas le fermer par `/>`.

until

Ne peut être placé que dans le bloc init et ne peut contenir que des affectations.

```
<until @a > 0>
@a = @__a._10 - 3
</until>
```

Répète les affectations **jusqu'à** ce que la condition soit vraie. La condition n'est évaluée qu'à la fin. Pertinent quand on veut intialiser une valeur aléatoire en respectant une condition sans pour autant tout redémarrer comme avec needed.

Pour éviter une situation bloquante, `until` n'est répété au plus que 100x.

Ainsi, si le professeur indique une condition trop contraignante ou impossible, la tentative d'exécution échoue et un message est affiché.

Les blocs constituant un exercice

Ces blocs ne seront présents que dans la zone de code, pas dans l'initialisation.

Forme générale

Déclaration

Un bloc est déclaré de la façon suivante :

```
<nombloc:titre>
...
</nombloc>
```

Certains type de blocs auront un comportement particulier comme les blocs `form`, `texte`, `input`, `graph`, ...

Un bloc peut contenir des sous blocs. Par exemple un bloc `form` peut contenir des blocs `input` ou `text`. Un bloc `graph` peut contenir des blocs `point` ou `function`...

Paramètres

Un bloc peut également recevoir des paramètres.

```
<nomparam:value/>
```

Bien noter le `/` final qui indique un bloc autofermant. Ce format de balise indique que l'on ajoute un paramètre au bloc.

Certains paramètres sont essentiels au fonctionnement. Par exemple :

```
<input:x>
  <tag:$x$/>
  <solution:3>
</input>
```

On renseigne ici que le bloc `<input>` aura un paramètre `solution` égal à 3 et une étiquette `$x$` qui sert à l'affichage.

La casse du nom de paramètre n'a pas d'importance. Ici, par exemple, vous pouvre aussi bien mettre `Solution` ou même `SOLUTION`.

Paramètres comme tableaux

Il est possible de traiter ces paramètres en tableaux :

```
<param[]:1/>
<param[]:7/>
```

Ainsi on a un paramètre de nom `param` qui est le tableau `[1, 7]`.

Toutefois, si on ajoute plusieurs valeurs dans un même param, celui-ci sera toujours compris comme un tableau.

```
<param:1/>
<param:7/>
```

Dans ce cas, le `7` n'écrase pas le `1`. La première ligne produit `param = 1` et la seconde ligne met à jour `param = [1, 7]`.

Un paramètre peut aussi être un texte avec formatage. Par exemple `<param:image de {@x:}/>`

Formatage

Dans une zone de texte, on est susceptible d'ajouter une formule. On peut alors écrire un bloc de formatage `{expr:}`. Ce bloc indique que l'on veut formater expression selon un certain formatage. Bien sûr on peut vouloir afficher le contenu d'une variable : `{@x:}`. Il existe divers formages.

- `{@x:}` pas de formatage particulier, résultat brut.
- `{@x:$}` formaté en latex non simplifié. Attention, ce formatage se contente de produire le code latex, par exemple `\times` pour une multiplication. Ce texte apparaîtra brut dans le champ de texte cible.

  Si on souhaite en plus que ce code soit identifié comme du latex et soit rendu en tant que tel, il faudra ajouter les `$`, donc écrire `${@x:$}`
- `{@x:s$}` est une version alternative pour le formatage en latex ajoutant une simplification légère. Ainsi les calculs évidents seront exécutés afin d'afficher correctement des cas comme `{@a*x^2+@b*x+@c:s$}` où l'un des coefficients serait nul ou égal à 1 ou -1.
- `{@x:f}` ou `{@x:3f}` pour un affichage approximé. Si on ne précise pas de nombre de chiffres après la virgules, alors le nombre est écrit en entier.

> Attention : nerdamer qui est utilisé pour les calculs remplace automatiquement les nombres décimaux en fractions. Ainsi, si on écrit `2.1`, on aura `21/10`. Cela peut être gênant dans une expression. Le format `float` permet alors de convertir les nombres fractionnaires en approximations à virgule.

- `{@x:f$}` pour float et latex. En effet, comme précisé dans la note ci-dessus, nerdamer convertit les nombres décimaux en fractions. Alors on est ennuyé si on veut un affichage avec nombres décimaux et en rendu latex. Ce format est là pour cela. On peut préciser un niveau d'approx, par exemple `{@x:3f$}`.
- `{@x:b}` pour un formatage brut, c'est à dire que `@x` est substitué (on prend sa valeur) mais on se contente de prendre sa version String sans chercher à faire un parse du contenu.

Supposons que l'on ait produit un trinome de forme $a \cdot x^2 + b \cdot x + c$ et que l'on ait stocké dans des variables `@a`, `@b`, `@c` les valeurs des coefficients, générés aléatoirement pour les besoins de l'exercice.

On pourrait croire qu'il suffit d'écrire `$@a x^2 + @b x + @c$` pour obtenir une représentation correcte du trinome.

Mais cette méthode est mauvaise car `@a` pourrait être égal à `1` et `@b` pourrait être nul ou négatif... C'est pour cette raison qu'il convient d'utiliser le formatage. L'expression sera convenablement interprétée par nerdamer qui produira une représentation adéquat. On pourra donc noter : `${@a x^2 + @b x + @c:$}$`

Il est enfin possible d'ajouter un calcul directement dans le formatage : `{1+3*5 -x:s}` commencera par parser l'expression et la simplifiera ce qui donnera `16-x`.

Utilisation de `expand`

Une difficulté est que nerdamer voudra probablement travailler la forme de l'objet numérique. Avec un trinome, il voudra automatiquement le développer. Que faire si nous désirons une forme développée ?

On peut utiliser la fonction `expand`. Avec l'exemple précédent, on pourra par exemple écrire :

`${expand(@a*x^2 + @b*x + @c):$}$`

L'expression est analysée de sorte que si `@b == 0`, le terme en `x` n'apparaîtra pas. `expand` force un affichage développé.

On peut aussi utiliser le format `s$` qui évite les mécanismes de nerdamer :

`${@a*x^2 + @b*x + @c:s$}$`

Bloc de texte

```
<texte:header>
...
</texte>
```

Le plus simple, sert à afficher du texte. `header` permet d'indiquer un titre mais n'est pas requis.

Au lieu de `texte` on dispose de quelques variantes :

- `texte` ou `text`, bloc ordinaire
- `warning` pour un formatage d'alerte
- `info` pour un formatage d'info
- `help` ou `aide` pour un bloc d'aide qui se replie. Il faut appuyer le bouton pour afficher l'aide.

On peut ajouter des éléments de formatage dans le texte :

- `un exemple de **texte** avec du gras`, ici `texte` sera en gras,
- `un exemple de __texte__ avec de l'italique`, ici `texte` sera en italique,
- `un pictogramme ::warning:: ici` et on aura un symbole warning. On a pour l'instant warning, info, exclamation, star, heart, bolt, brain, calc, note, idea
- `(!warning!) au début de la ligne`, le code en début de ligne demande un formatage type warning du paragraphe. On a également info, danger, success.

Bloc formulaire

```
<form:header>
...
</form>
```

Ce bloc est essentiel pour le déroulement des exercices car c'est lui qui prend en charge les questions / réponses.

En effet, quand l'exercice s'affiche, il affiche les blocs dans l'ordre et quandd il arrive à un formulaire il s'arrête en attendant que l'utilisateur réponde aux questions. Quand l'utilisateur a répondu, l'affichage se met à jour et l'exercicce se poursuit.

S'il s'agit d'un exercice qui a déjà été exécuté et que l'on rejoue, les réponses sont déjà données. Le formulaire le détecte et affiche directement la suite.

Un formulaire contiendra donc des input divers et des blocs de texte.

Bloc input

Il s'agit de l'input de base.

```
<input:name>
...
</input>
```

Un champ input se traduira par une zone de saisie dans laquelle on peut entrer du texte. Les paramètres sont ici très importants, il faut détailler.

- header : c'est l'identifiant de la réponse telle qu'elle sera sauvée dans la BDD. Il faut donc que tous les input en ait un différent. De plus, cet identifiant ne devrait pas rentrer en conflit avec ceux déjà déclaré avec `@` dans l'initialisation et les options.
- tag : ce qui sera affiché pour représenter la valeur demandée. Un affichage Tex est possible. Par exemple x_A.
- solution : la valeur attendue. Il est possible de fournir un tableau. Dans ce cas, la réponse sera considére valide du moment qu'elle correspond à au moins un item de solution.
- `tagSolution` : on peut désirer forcer un affichage de solution. On peut le faire avec tagSolution. Voici un exemple d'usage :
- `placeholder` : dans un champ de saisie, on peut donner une indicatiojn en griser. Par défaut ce sera "Entrez votre réponse". Mais on peut choisir autre chose. Par exemple, si on veut inviter l'élève à entrer une équation de droite, on peut mettre "y=mx+p"

```
@f = (x+5)*(x+7)
<form>
  <input:d>
    <tag:Développer ${@f:$}$/>
    <solution:@f/>
    <vars:x:expand/>
    <keyboard:square/>
    <tagSolution:${expand(@f):$}$/>
  </input>
</form>
```

Format

Le paramètre format permet d'indiquer le format attendu.

Voici les formats reconnus :

- `numeric` : on veut une expression dans laquelle il n'y a aucune variable et cette expression doit être développée un minimum. On n'acceptera pas `(1+2)/3`.
- `round:2` : on veut un nombre arrondi, dans cet exemple à deux chiffres après la virgule.
- `entier` ou integer, équivalent à `round:0`
- `erreur:0.2` : demande un nombre. Sa valeur ne doit pas excéder une erreur de 0.2 dans l'exemple.
- `empty` : indique que l'on accepte l'ensemble vide. On peut alors répondre par `'vide'` ou `'∅'`
- infini : indique que l'on accepte un infin comme `'+inf'` ou `'-∞'`. Le signe est obligatoire même pour +
- `expand` : indique que l'on attend une expression développée.
- `var:x` : indique que l'expression peut dépendre de x. on pourrait écrire `var:xn` pour autoriser une expression dépendant de `x` et de `n`.
  Il faut fournir un résultat simplifié (par ex, les calculs de nombre comme 2+3 ou 2x+3x doivent être simplifiés)
- `var:x:expand`, comme le précédent mais en indiquand que l'on veut forcer le développement
- `equation:xy` : indique que l'on attend une équation linéaire dépendant de x et y comme $3x + 2y - 5 = 0$. Dans ce cas la solution ne devra être qu'une expression membre gauche = 0. Par ex si l'équation est possible `3x = 4` la solution aura la forme `3x-4`, sous entendu = 0.
- `vecteur:2:colinear:notnul` le 2 est la taille (il faut au moins 2), "colinear" et "notnul" sont optionnels. "colinear" indique que l'on accepte un vecteur colinéaire et "notnul" indique que l'on acceptera pas un vecteur nul.
- `criterion:vars:expression` : il s'agit d'un critère général. Par exemple `criterion:xy:x+y==0` acceptera une paire `x;y` dont la somme est 0, comme `2;-2`. Dans l'expression, on pourra ajouter `x0`, `y0` pour prendre les valeurs de la solution proposée.

Il ne serait pas logique de proposer à la fois numeric et round mais on peut demander empty et numeric par exemple.

On pourra donc utiliser : `<format:numeric/>` dans le cas où on veut permettre divers types de réponses :

```
<format:numeric/>
<format:empty/>
```

Les formats `infini` et `empty` auront également pour effet d'ajouter un bouton adapté au input.

Clavier

keyboard: permet d'ajouter un bouton pour des expressions mathématiques comme `sqrt`

- power
- sqrt
- square
- cube
- infini
- empty

Ainsi un bloc typique serait :

```
<input:x>
<tag:$x$/>
<solution:@gx/>
<format:numeric/>
<keyboard:sqrt/>
<keyboard:power/>
</input>
```

Dans ce cas on ajoute deux boutons, un pour racine et l'autre pour la puissance.

Si vous ajoutez un bloc d'aide dans un input, alors il y aura un bouton d'aide directement dans le input.

les keyboard `infini` et `empty` sont ajoutés automatiquement si on a précisé les format infini et empty.

On peut aussi ajouter son clavier custom :

```
<keyboard:name:tag:replacement/>
```

- `name` est un identifiant unique
- `tag` est ce qui apparaîtra sur le bouton
- `replacement` est le texte qui sera placé au niveau du curseur dans le champs
- `pre` et `post` : servent si l'utilisateur à sélectionner du texte et appuie le bouton

Quelques exemples :

```
<keyboard:exp/>
```

Dans ce cas, le bouton est étiqueté `exp` et le bouton ajoutera `exp` dans le champ de saisie.

```
<keyboard:exp:$e^x$/>
```

Dans ce cas, le bouton est étiqueté `$e^x$` et le bouton ajoutera `exp` dans le champ de saisie.

```
<keyboard:exp:$e^x$:e^x/>
```

Dans ce cas, le bouton est étiqueté `$e^x$` et le bouton ajoutera `e^x` dans le champ de saisie.

```
<keyboard:exp:$e^x$:e^x:exp(:)/>
```

Dans ce cas, le bouton est étiqueté `$e^x$`. Si aucun caractère n'est sélectionné dans le champ de saisie, alors le bouton ajoute `e^x`. Si on avait préalablement sélectionné par exemple les caractères `3t` dans le champ de saisie, le bouton remplace par `exp(3t)`

Pas de formule en solution !

Attention : n'essayez pas de mettre directement une formule comme solution. Le mieux est d'affecter au préalable une variable contenant la solution.

```
@sol = ...
<input:a>
...
<solution:@sol/>
...
</input>
```

excluded

On a le droit d'exclure une valeur. Par exemple si on avait une droite passant par un point A et que l'on voulait les coordonnées d'un point de cette droite, il serait naturel de refuser les coordonnées de A.

```
<excluded:3;4/>
```

On pourrait exclure plusieurs valeurs :

```
<excluded[]:3;4/>
<excluded[]:2;6/>
```

Bloc Radio

Utiliser pour une question à choix multiple. Une seule réponse possible.

Voici un exemple :

```
<radio:n>
0 => aucune solution
1 => une solution
2 => deux solutions
<solution:@gn/>
</radio>
```

Ici `@gn` qui aura été préalablement initialisé, contiendra une valeur parmi 0, 1, 2.

À l'affichage, les différentes possibilités sont mélangées, mais bien sûr, chaque réponse reste convenablement associée à sa clé.

- le paramètre `solution` est obligatoire
- vous disposez du paramètre `tag` qui sert à améliorer l'affichage de la réponse. Par ici, on pourrait même le tag `<tag:Nombre de solutions>` de sorte que si l'élève répond "deux solutions", on pourra avoir un affichage comme "Nombre de solutions : 2 solutions est une bonne réponse".

Bloc input ensemble

Ci-dessous, deux exemple avec les solutions.

```
<form>
<inputensemble:a>
<tag:La réponse est [2;4]/>
<solution:[2;4]/>
</inputensemble>
<inputensemble:b>
<tag:La réponse est [2;4]union[10;15]/>
<solution:[2;4]union[10;15]/>
</inputensemble>
```

L'utilisateur se verra proposer des boutons pour l'infini, vide et union. Il devra écrire un ensemble valide, dans l'ordre, sans recouvrement. Des intervalles comme $[10;12]\cup[11;13]$ pourtant valides mathématiquement ne seront pas admis.

Bloc Liste de choix

```
<choix>
1 => Le choix 1
2 => Le choix 2
3 => Le choix 3
</choix>
```

Ce bloc permet d'afficher une liste d'items associés à un pictogramme coloré dépendant de l'index indiqué. L'index doit être entre 1 et 7 inclus.

On dispose des paramètres :

- `shuffle:true` qui permet de mélanger à l'affichage (pas mélangé par défaut)
- `onlysquares:true` pour demander à n'avoir que des carrés. Par défaut true.

Les pictogrammes peuvent servir pour les daltoniens. On peut donc choisir de ne pas utiliser les pictogrammes.

Formulaire liste de choix

```
<inputchoix:name>
1 {label} => valeur 1
2 => valeur 2
1 => valeur 3
</inputchoix>
```

`label` est optionnel.

Il s'agit d'un input. `name` est l'identifiant de la réponse. Il faut le placer dans un bloc

Cet input se présentera comme une liste de choix avec des pictogrammes colorés. Chaque item est un bouton. Quand on clique, on change le pictogramme et la couleur.

Voyons un exemple d'utilisation :

```
<choix>
1 => $\mathcal{N}$
2 => $\mathcal{Z}$
3 => $\mathcal{D}$
</choix>

<form>
<inputchoix:a>
2 => -5
1 => 8
1 => 0
3 => 5,4
2 => -1
</inputchoix>
</form>
```

Le premier bloc définit les couleurs (et donc indices) associées aux ensembles. On demande ensuite à l'élève de choisir. Il devra choisir 2 pour -5, 1 pour 8, etc.

On peut préciser le param `max:4/ pour indiquer que l'on autorise le choix à aller jusque 4 même si ce n'est pas une des réponses prévues.

Remarque : les indices peuvent se répéter dans `inputchoix` car plusieurs items peuvent avoir la même réponse. Ici, 8 et 0 sont tous deux des entiers naturels.

On dispose là encore de paramètres :

- `shuffle:false` pour le mélange (mélangé par défaut)
- `onlysquares:true` pour ne pas utiliser les pictogrammes
- `shuffleindexwalk:true` dans certain cas, on aura les items toujours dans le même sens. Par exemple j'ai un exercice avec trois A, B, C à reconnaître et je construis mon exercice de façon fixe de sorte que A est toujours la première des couleurs, B la seconde... Cela peut-être ennuyeux. Alors on peut forcer un parcours des valeurs aléatoire lors des clics de sorte que le A a beau toujours avoir la valeur 1, B 2 et C 3, ça ne se voit pas car les index sont parcourus dans un ordre quelconque.

Cas d'une réponse calculée

Supposons que l'exercice calcule une variable `@g` qui vaut `1` ou `2` selon le cas. Vous pouvez faire :

```
<choix>
1 => Le choix 1
2 => Le choix 2
</choix>
<form>
<inputchoix:a>
@g => Choisir la bonne réponse
</input>
</form>
```

On pourrait aussi procédéer ainsi

```
<form>
<inputchoix:a>
1 {tag} => $\mathbb{N}$
2 {tag} => $\mathbb{Z}$
3 {tag} => $\mathbb{D}$
2 => -5
1 => 8
1 => 0
3 => 5,4
2 => -1
</inputchoix>
</form>
```

Les trois lignes `{tag}` permettent de préciser l'étiquette correspondant à 1, 2 et 3.

Ainsi, on aura directement des étiquettes au lien des carrés de couleurs ou des pictogrammes pour les choix et il n'est pas nécessaire de faire précéder d'un bloc choix.

Cela peut être plus lisible selon les cas et permet d'éviter l'usage de couleurs qui pourraient handicaper des élèves qui ne distinguent pas les couleurs.

Bloc Table

```
<table>
...
</table>
```

Permet d'afficher une table.

Il faudra lui fournir un paramètre `rows` qui est un tableau de tableaux.

On pourra lui fournir des tableaux `rowheaders` et `colheaders`.

Voici un exemple d'initialisation.

```
# init
# tableau a = [0, ..., 9]
@a <:10>= @__i
# tablau b = [a[i]*2 + alea]
@b <:@a>= <P:@__v 2 * 0 5 Alea.entier +>

# affichage de la table
<table>
<rowheaders[]:$x$/>
<rows[]:@a/>
<rowheaders[]:$f(x)$/>
<rows[]:@b/>
</table>
```

Bloc tkztab

Il s'agit d'un bloc pour tableau de variations ou de signe, nommé ainsi en relation avec le module tikz latex tkz-tab dont j'ai repris certaines notations.

```
<tkztab>
<color:red/>
<xlist:$1$,$a$,3,$+\infty$/>
<sign:$A$:1:,+,z,-,z,+,/>
<sign:$f(x)$:2:,-,d,+,z,-/>
<var:$f(x)$:3:-/1,+D-/$x+3$/$-\infty$,+/0,-/$-\infty$/>
</tkztab>
```

D'abord quelques paramètres :

- `xlist` est **obligatoire**. C'est un tableau ou un texte indiquant les valeurs des antécédents. On peut mettre des éléments latex qui seront transformés par katex. Naturellement, comme pour tous les params, on a le droit de placer des variables avec un formatage comme `${@a:$}$`.
- `color` pour choisir la couleur du tableau. On peut utiliser une couleur html standard comme `red` ou un numéro ce qui permet de faire le liens avec le bloc de choix.
- `tag` pour choisir l'entête de la première ligne. Par défaut, c'est `$x$`
- `espcl` pour choisir l'espacement entre les items, en pixels, dans la partie droite du tableau. 150 par défaut.
- `lgt` pour choisir la largeur du bloc d'entête, 100 par défaut
- `pixelsperline` pour choisir la hauteur d'une unité verticale, 40 par défaut
- `headerHeight` pour choisir la hauteur de la ligne d'entête, exprimée en unité

Noter que le SVG s'ajuste à l'espace disponible en largeur.

Ensuite viennent les contenus à proprement dit.

Ligne tableau de signe

La forme est `<sign:$f(x)$:2:,-,d,+,z,-,/>`

- `sign` indique qu'il s'agit d'une ligne tableau de signe

- `$f(x)$` est l'entête (obligatoire)
- `2` est la hauteur de la ligne, en unité (obligatoire, minimum 1)
- vient ensuite le contenu `,-,d,+,z,-,` qui respecte la syntaxe tkz-tab

La ligne sera tronquée ou complétée en fonction de la taille attendue avec le `xList` du parent. Si par exemple xList a 3 items, on aura donc 3 valeurs de x et deux intervalles, donc la ligne devrait avoir 5 éléments. Les éléments de rangs pairs correspondent aux valeurs de x et ceux de rang impair correspondent aux intervalles, donc aux signes.

- un signe peut être `+` ou `-` (autre ignoré)
- en face d'un x, on peut avoir `z` pour zéro, `d` pour interdit et `t` pour une cloison neutre (autre ignoré)

Naturellement, comme pour tous les params, on a le droit de placer des variables avec un formatage comme `${@a:$}$`.

input ligne tableau de signe

La forme est `<inputsign:$f(x)$:2:,-,z,-,:b/>`

- `inputsign` indique qu'il s'agit d'une ligne tableau de signe input
- `$f(x)$` est l'entête (obligatoire)
- `2` est la hauteur de la ligne, en unité (obligatoire, minimum 1)
- `,-,z,-,` est la réponse attendue
- `b` est l'identifiant de la réponse, utilisée notamment pour la sauvegarde BDD

Au moment de poser la question, tous les signes seront mis à `+` mais les bornes seront celles de la solution.

Un tableau contenant une ligne *inputsign* devrait être placé dans un bloc .

Rien n'empêche d'avoir plusieurs lignes input dans un même tableau.

Ligne tableau de variation

Exemple `<var:$f(x)$:3:-/1,+D-/$x+3$/$-\infty$,+/0,-/$-\infty$/>`

- `var` signale que c'est une ligne de variations
- `$f(x)$` est l'entête (obligatoire)
- `3` est la hauteur en unités (obligaoire, minimum 3)
- vient ensuite le contenu `+D-/$x+3$/$-\infty$,+/0,-/$-\infty$` qui respecte la syntaxe tkz-tab.

Le nombres d'items de la ligne doit correspondre à la taille du `xList` parent. Il est complété ou tronqué au besoin.

Les items autorisés ont toujours la forme : `pos/tag/tag`. Les tags sont optionnels et selon les cas, une seul ou deux sont prises en compte.

Chaque fois, `pos` permet d'indiquer la forme. On a les cas de tkz-tab :

- `-`, cas normal, en bas
- `+`, cas normal, en haut
- `-D`, valeur interdite avec une valeur en bas sur la gauche

- `+D`, valeur interdite avec une valeur en haut sur la gauche
- `D-`, valeur interdite avec une valeur en bas sur la droite
- `D+`, valeur interdite avec une valeur en haut sur la droite
- `-D-`,
- `-D+`,
- `+D-`,
- `+D+`,
- `R`, position igorée

Naturellement, comme pour tous les params, on a le droit de placer des variables avec un formatage comme `${@a:$}$`.

input ligne tableau de variation

On peut ajouter une ligne de tableau de variations avec des boutons permettant de choisir la position des flèches.

Exemple `<inputvar:$f(x)$:3:+,+D+,+:a:-,+D-,+/>`

- `inputvar` signale que c'est une ligne de variations input
- `$f(x)$` est l'entête (obligatoire)
- `3` est la hauteur en unités (obligaoire, minimum 3)
- `+,+D+,+` est la position des flèches telle qu'elle apparaîtra au moment de poser la question. On est libre de placer des valeurs mais c'est plus lisible sans.
- `a` est le nom de la réponse telle qu'elle sera stockée en BDD
- `-,+D-,+` est la réponse attendue. On peut ajouter des valeurs comme `-/$0$,+D-/$+\infty$/$-\infty$,+/$0$`. Ces valeurs seront de toute façons ignorées (l'utilisateur ne peut les changer) Ces valeurs peuvent enrichir la réponse.

Un tableau contenant une ligne *inputvar* devrait être placé dans un bloc .

Rien n'empêche d'avoir plusieurs lignes input dans un même tableau.

Bloc Graph

Permet de tracer un graphique avec **jsxGraph** et peut être ajouter comme input.

```
<graph>
...
</graph>
```

Le bloc a lui-même quelques paramètres :

- xmin: bord gauche, par défaut à -5
- xmax: bord droit, par défaut à +5
- ymin: bord bas, par défaut à -5
- ymax: bord haut, par défaut à +5

- zoom: autorise ou non le zoom. Par défaut à false.
- pan: autorise ou non le déplacement de la vue. Par défaut à false.
- axis: affiche ou non les axes. Par défaut à true.
- grid: affiche ou non la grille. Par défaut à true.
- aspectratio: proportions du graphes. Par défaut 1/1, donc un carré. 2/1 par ex donnera un rectangle horizontal 2x plus large que haut.
- gridX et gridY: pas de grille

Ensuite, on peut ajouter des sous-blocs pour les différents objets.

Attention ! Le nom donné à un objet dans sa déclaration, comme `<point:A>` ne préjuge pas de l'étiquette qui sera attribuée par l'outil graphe. Le graphe attribut les étiquettes dans l'ordre. Pour attribué une étiquette déterminée, utilisez l'attribut name : `<name:A/>`. Cela laisse la liberté d'utilser des caractères UTF8.

point

```
<point:name>
...
</point>
```

On dispose des paramètres :

- x
- y
- name: pour l'affichage d'un nom
- on: alors le point est un glider qui glisse sur un objet. Il faut choisir le nom d'un objet graphique défini préalablement.
- color: on peut choisir une couleur standard comme blue, red... ou un indice ce qui choisira parmi les couleurs des blocs choice
- size: pour la taille du point
- fixed: par défaut le point est mobile. On peut le fixer avec `fixed:true`
- labelsize: taille de l'étiquette, par défaut 14
- forme: forme du point (parmi cross, circle, square, plus, divide, diamond, triangleup, triangledown, triangleleft, triangleright), par défaut circle
- solution:true l'élément ne sera visible qu'au moment de la solution
- distinct:A,B;0.5 indique que l'élément doit-être distinct d'un autre avec la distance min

Le point peut par ailleurs être un input. On ajoutera pour cela des attributs :

- hasinputs: il faut les écrire sous la forme x->xA;y->yA
  point n'admet que les attributs x et y
- good: pour indiquer le critère d'acceptation
  - pourra prendre la forme (2;4;0.1) par exemple pour un point avec une distance max de 0.1
  - ou encore (2;4) la distance max étant fixée alors à 0.1 par défaut
  - ou encore 3x+2y-5, qui est un critère devant valoir 0 avec une erreur de 0.1 par défaut

- ou encore 3x+2y-5;0.2 pour fixer l'erreur à 0.2

axe des réels

```
<reels:name>
<y:5/>
<titlesize:15/>
</reels>
```

- y: décalage y
- titlesize: taille du titre $\mathbb{R}$ (défaut 20)
- tikssize: taille des étiquettes (défaut à 12)
- color: couleur de l'axe
- majorheight: hauteur des ticks des entiers (défaut 10)
- minorheight: hauteur des ticks des subdivisions (défaut 5)
- minorticks: nombre de ticks subdivision entre deux entiers (défaut 4 pour 5 intervalles)
- strokewidth: épaisseur du trait (défaut 1)

function ou fonction

```
<function:name>
...
</function>
```

ou bien en français

```
<fonction:name>
...
</fonction>
```

On dispose des paramètres :

- expression: l'expression de la fonction
- xmin: par défaut le bord gauche de la fenêtre
- xmax: par défaut le bord droit de la fenêtre
- color: couleur du trait
- strokecolor: même chose
- strokewidth: épaisseur du trait
- dash: pointillé. Par exemple `dash:2`
- solution: true, l'élément ne sera visible qu'au moment de la solution

Voici deux exemples : Dans le premier on crée la courbe d'une fonction affine et un point mobile sur cette courbe.

```
<function:f>
<expression:3x+12/>
</function>
<point:A>
<on:f/>
</point>
```

Dans ce 2e exemple, on crée une expression aléatoire d'un polynome de degré 3.

La fonction `Alea.lagrangePolynome` tire `n+1` points au hasard, à coordonnées entières et dans le rectangle défini par xmin, ymin, xmax, ymax. Avec ces points elle calcule le polynome d'interpolation de Lagrange.

```
@p = <P:-2 -5 5 5 3 Alea.lagrangePolynome>
<graph>
  <xmin:-2/>
  <function:g>
    <expression:@p/>
  </function>
</graph>
```

droite

```
<droite:name>
...
</droite>
```

On dispose des paramètres :

- equation: l'expression de la droite sous le mode désiré :
    - réduite en $y = 2x + 3$ ou $x = 5$
    - cartésienne $2x + 3y - 11 = 0$
    - toute autre forme $3x = 5y - 4$
- points: 2 points par lesquelles passe la droite
    - on peut donner un identifiant de point déjà créé
    - ou une paire de coordonnées comme (2.5;4)
    - on peut faire un mélange (objet point et coordonnées)
    - séparer les deux points par |
- color, idem que pour point
- strokewidth: épaisseur du trait
- dash: pointillé. Par exemple `dash:2`
- solution: true, l'élément ne sera visible qu'au moment de la solution
- label: l'étiquette nommant la droite
- labelsize: taille de l'étiquette (défaut 14)
- invisible: true, pour rendre la droite invisible

Voici quelques exemples.

```
<droite:d>
<equation:y=3x+12/>
</droite>
```

```
<point:A>
  <x:2/>
  <y:5/>
</point>
<droite:d>
  <points:A|(0;1)/>
  <color:red/>
</droite>
```

intégrale

```
<integrale:name>
...
</integrale>
```

Trace l'aire sous une courbe et fait le calcul approximatif de l'aire. Place automatiquement deux curseurs pour modifier les bornes gauche et droite.

On dispose des paramètres :

- abscisses: deux valeurs numériques séparées par |. En l'absence de ce paramètre, les bords du cadre seront choisis.
- fct: le nom de l'objet fonction qui doit avoir été défini avant.
- fixleft:true, permet de fixer la borne gauche
- fixright:true, permet de fixer la borne droite
- fixed:true, permet de fixer les deux bornes
- hidelabel:true, permet de masquer le texte de résultat de l'intégrale
- color: couleur, comme pour point. Par défaut, rouge.
- solution:true, indique que l'objet graphique n'apparaîtra qu'après validation du formulaire

Voici un exemple :

```
<function:g>
  <expression:exp(-x^2/2)/>
</function>
<integrale:i>
  <abscisses:-1|1/>
  <fct:g/>
  <fixleft:true/>
  <color:blue/>
</integrale>
```

Polygone

```
<polygon:name>
...
</polygon>
```

ou en français

```
<polygone:name>
...
</polygone>
```

On dispose des paramètres :

- points: liste de points séparés par |
  chacun de ces points pourra être
    - soit un nom de Point déjà défini
    - soit une paire de coordonnées (x;y)
- fixed:true, permet de fixer les points donnés avec des coordonnées
- color, couleur de remplissage. Si pas défini, pas de remplissage
- opacite ou opacity: exprimé en %, opacité de l'intérieur
- strokewidth: épaisseur du trait (défaut 1) Pour ne pas avoir de trait, mettre 0
- strokecolor: couleur du trait (par défaut, comme color sinon black)
- solution: true, l'élément ne sera visible qu'au moment de la solution
- dash: pointillé
- open:true ou ouvert:true, pour une ligne polygonale ouverte

Angle

```
<angle:name>
...
</angle>
```

On dispose des paramètres :

- name: pour le label. On peut utiliser, pour les lettres grecques, des code html comme `α` ou `Α` pour les majuscules.
- points: liste de points séparés par |, il en faut 3
  chacun de ces points pourra être
    - soit un nom de Point déjà défini
    - soit une paire de coordonnées (x;y)
- fixed:true, permet de fixer les points donnés avec des coordonnées
- color, couleur de remplissage. Si pas défini, pas de remplissage
- showvalue:true, indique ou pas la valeur de l'angle

- solution: true, l'élément ne sera visible qu'au moment de la solution
- ortho:true, pour marquer l'angle droit le cas échéant

segment

```
<segment:name>
...
</segment>
```

On dispose des paramètres :

- points: liste de points séparés par |, il en faut 2
  chacun de ces points pourra être
    - soit un nom de Point déjà défini
    - soit une paire de coordonnées (x;y)
- fixed:true, permet de fixer les points donnés avec des coordonnées
- strokecolor ou color, couleur du trait (défaut black)
- strokewidth: épaisseur du trait (défaut 1)
- solution: true, l'élément ne sera visible qu'au moment de la solution
- dash: pointillé

vecteur

```
<vector:name>
...
</vector>
```

Le mot clé `vecteur` est accepté également.

On dispose des paramètres :

- points: liste de points séparés par |, il en faut 2 ou 1.
  chacun de ces points pourra être
    - soit un nom de Point déjà défini
    - soit une paire de coordonnées (x;y)
      Dans le cas où il y a deux points, représentent les extrémités.
      Dans le cas où il n'y a qu'un point, `points` fonctionne comme `coords`
- coords: une paire de coordonnées qui représentes les coordonnées du vecteur.
- start: coordonnées ou point représentant l'origine. par défaut (0;0)
- fixed:true, permet de fixer le vecteur
- strokecolor ou color, couleur du trait (défaut black)
- strokewidth: épaisseur du trait (défaut 1)
- solution: true, l'élément ne sera visible qu'au moment de la solution
- dash: pointillé
- showsend:true, montre un point pour l'extrémité, ce qui permet de modifier les coordonnées du

vecteur.

Le vecteur est conçu pour ne pas être lié à des points. Donc, si on crée par exemple un vecteur $\overrightarrow{AB}$ basé sur des points A et B déjà créés, le vecteur ne sera pas lié à eux.

Bloc shuffle

À l'exécution, mélange ses enfants et renvoie ces enfants. Peut servir par exemple si on veut produire des tableaux de variations et les mélanger.

```
<shuffle>
...
</shuffle>
```

Les fonctions disponibles

Module Alea

- `Alea.entier` reçoit `xmin` et `xmax` et renvoie un entier aléatoire entre `xmin` et `xmax` compris.
- `Alea.decimal` reçoit `xmin` et `xmax` et renvoie un décimal (float) aléatoire entre `xmin` (compris) et `xmax` (exclu).
- `Alea.signe` renvoie `-1` ou `1` aléatoirement
- `Alea.lagrangePolynom` reçoit `xmin`, `ymin`, `xmax`, `ymax` et `n`. Renvoie un polynome de degré `n` interpolant `n+1` points de coordonnées entières pris dans le cadre.
- `Alea.choice`, reçoit un tableau et renvoie un des items du tableau au hasard
- `Alea.premier`, reçoit un entier `n` et renvoie un premier `<= n`. Attention, `n` peut aller de 2 à 1008 (la liste prédéfinie de premiers va de 2 à 997)

Module table

- `Table.get` reçoit `tableau` et `indice`. Renvoie l'élément à l'indice donné.
- `Table.indice` reçoit `val` et `tableau`. Renvoie l'indice de la première occurence de `val` dans `tableau`. `-1` si absent.
- `Table.indices` reçoit `val` et `tableau`. Renvoie un tableau contenant tous les indices des occurences de `val` dans `tableau`
- `Table.size` reçoit `tableau` et renvoie la taille du tableau.
- `Table.sum` renvoie la somme des éléments du tableau
- `Table.product` reçoit `tab1` et `tab2` qui doivent être de même taille et renvoie un tableau des `it1*it2`
- `Table.average` reçoit `tableau` et renvoie la moyenne des éléments du tableau. Erreur si tableau vide.
- `Table.average2` reçoit les tableaux `values` et `effectifs`, de même taille, et renvoie la moyenne.
- `Table.covariance` reçoit `tableau1` et `tableau2` de même taille et renvoie la covariance des éléments des deux tableaux.
- `Table.variance` reçoit `tableau` et renvoie la variance des éléments du tableau. Erreur si tableau vide.

- `Table.variance2` reçoit les tableaux `values` et `effectifs`, de même taille, et renvoie la variance.
- `Table.std` reçoit `tableau` et renvoie l'écart-type des éléments du tableau. Erreur si tableau vide.
- `Table.std2` reçoit les tableaux `values` et `effectifs`, de même taille, et renvoie l'écart-type.
- `Table.mediane` reçoit `tableau` et renvoie la médiane des éléments du tableau. Erreur si tableau vide.
- `Table.mediane2` reçoit les tableaux `values` et `effectifs`, de même taille, et renvoie la médiane.
- `Table.quantile` reçoit `tableau` et `q` et renvoie la quantile `q` des éléments du tableau. Erreur si tableau vide.
- `Table.quantile2` reçoit les tableaux `values` et `effectifs`, de même taille, et `q`, et renvoie la quantile.
- `Table.ECC2` reçoit les tableaux `values` et `effectifs` et une `valeur`. Renvoie l'effectif total de tous les individus dont la valeur est inférieure ou égale à `valeur`
- `Table.min` reçoit un tableau et renvoie la valeur minimum.
- `Table.max` reçoit un tableau et renvoie la valeur maximum.
- `Table.sortFreqs` reçoit un tableau et renvoie un tableau de tableau `t` avec `t[0]` contenant la liste des valeurs en ordre croissant et `t[1]` la liste des effectifs.
- `Table.filter` reçoit un tableau, un opérateur parmi `==, !=, <, <=, >, >=`, et une valeur. Renvoie le tableau des items satisfaisant le test.
- `Table.toBrut` reçoit les tableaux `valeurs` et `effectifs` et reconstitue la série brute, dans l'ordre.
- `Table.sort` reçoit un tableau `valeurs` et renvoie une copie triée.
- `Table.shuffle` renvoie une copie mélangée d'un tableau `valeurs`

Module Calc

- `*` fait une multiplication. Attention, en notation polonaise inversée donc par ex `<P:3 5 *>` pour faire 3×5
- `-`, `+`, `/` : idem
- `pow` ou `^` ou `**` pour exponentiation
- `div` pour la division entière
- `mod` pour le modulo
- `pgcd` pour le PGCD
- `ppcm` pour le PPCM
- `float` pour obtenir un float
- `abs` pour valeur absolue
- `exp` calcule l'exponentielle
- `sqrt` calcule la racine carrée
- `factorial` calcule la factorielle
- `sign` renvoie `-1` pour un négatif et `+1` pour un positif
- `solve` reçoit `left`, `right` (deux membres de l'équation) et `name` nom de la variable. Renvoie les solutions

- `sub` reçoit `expression`, `name` et `value` et renvoie l'expression où on substitué `name` pour `value`. Permet en particulier de calculer $f(x)$.
- `round` reçoit une valeur `x` et un `entier`. Arrondi à n digits après la virgule (n peut être <0)
- `floor` comme `round` mais à l'inférieur
- `ceil` comme `round` mais au supérieur
- `expand` développe l'expression. *Attention: pour l'instant, expand utilise forcément nerdamer ce qui crée pour certains calculs des affichages peu sympathiques, donc faire attention si vous voulez afficher le résultat d'un expand*
- `derivate` ou `deriver` dérive l'expression selon sa variable qui devrait être unique
- `diff` dérive l'expression selon la variable fournie en second argument
- `max` reçoit `x` et `y` et renvoie le plus grand des deux
- `min` reçoit `x` et `y` et renvoie le plus petit des deux
- `simplify` reçoit `x` et en renvoie une version simplifiée
- `taylor`, reçoit une expression, une variable et un ordre (0 à 5) et renvoie l'expression du développement limité en 0 de l'expression, à l'ordre demandé.

Module Dist

- `Dist.bincoef` reçoit les entiers `n` et `k` et renvoie $\binom{n}{k}$.
- `Dist.binomial` reçoit `n` et `p` et simule un aléa $\mathcal{B}(n;p)$
- `Dist.binList` reçoit `count`, `n` et `p` et renvoie un tableau de `count` simulations de $\mathcal{B}(n;p)$
- `Dist.binCDF` reçoit `x`, `n` et `p` et renvoie la probabilité $p(X \leqslant x)$ avec X suivant $\mathcal{B}(n;p)$
- `Dist.binPDF` reçoit `k`, `n` et `p` et renvoie la probabilité $p(X = k)$ avec X suivant $\mathcal{B}(n;p)$
- `Dist.poissonCDF` reçoit `x` et `lambda` et renvoie la probabilité $p(X \leqslant x)$ avec X suivant $\mathcal{P}(\lambda)$
- `Dist.binPDF` reçoit `k`, `lambda` et renvoie la probabilité $p(X = k)$ avec X suivant $\mathcal{P}(\lambda)$
- `Dist.normCDF` reçoit `x`, `mu` et `std` et renvoie la probabilité $p(X \leqslant x)$ avec X suivant $\mathcal{N}\left(\mu\,;\sigma^2\right)$
- `Dist.normPDF` reçoit `x`, `mu` et `std` et renvoie la valeur de la fonction densité en x pour la loi $\mathcal{N}\left(\mu\,;\sigma^2\right)$
- `Dist.normal` reçoit `mu` et `std` et renvoie un tirage selon la loi $\mathcal{N}\left(\mu\,;\sigma^2\right)$
- `Dist.normList` reçoit `count`, `mu` et `std` et renvoie un tableau de `count` aléas selon la loi $\mathcal{N}\left(\mu\,;\sigma^2\right)$.

Module Str

- `Str.replace` reçoit `expression`, `searchValue` et `replaceValue`. Permet le remplacement. Fonctionne si `expression` est un tableau.
- `Str.prefix` reçoit `expression` et `prefix`. Ajoute le prefix à expression. Fonctionne si expression est un tableau.
- `Str.postfix` reçoit `expression` et `postfix`. Ajoute le postfix à expression. Fonctionne si expression est un tableau.

Module Tkztab

- `sign` reçoit une `expression` et un tableau de `bornes`. Renvoie la chaîne représentant la ligne de signes comme dans un tableau TkzTab, en fonction des bornes. Il faut que les bornes correspondent car la fonction se contentera de calculer selon les bornes et aux milieux entre les bornes.
- `ensemble` reçoit `expression`, `bornes` et `asked`. Il faut avoir préparé les bornes du tableau de signe pour qu'elles correspondent à l'expression. On calcule les signes comme pour le tableau de signe. `asked` est égal à `p` (positif) ou `p0` (positif ou nul) ou `n` ou `n0`. Par exemple, si c'est `p` on renvoie sous forme d'une chaîne l'ensemble solution de `expression > 0`.